Comment diffuser

> **Note terminologique.** Cette page décrit la phase **DIFFUSER** dans son contexte opérationnel. Pour les définitions de référence des termes comme diffusion, build candidat, artefact, package, déploiement, rollback, release, validation, non-régression, couverture, contrat d'intégration, observabilité, Go / No-Go, CARE, rôles et RACI, voir le Lexique / Terminologies. Les contributions indiquées dans cette page sont propres à cette phase et ne remplacent pas la matrice RACI.

🎯 Objectif de la phase DIFFUSER

La phase DIFFUSER est le **pont entre FORGE et CARE** : elle reçoit de IMPLÉMENTER un code déjà conforme aux stories prêtes issues de PLANIFIER, puis elle assemble, valide, prouve, package, déploie, communique et transfère le produit pour le rendre disponible de manière maîtrisée. DIFFUSER ne découpe pas les stories et ne redéfinit pas le scope : elle exécute le plan de diffusion, vérifie les preuves et prépare la confirmation terrain.

> « Diffuser, ce n'est pas juste faire un package ou publier un build. C'est rendre réel tout ce qu'on a prévu — prouver que le produit est conforme, stable, déployable, documenté, supportable et prêt à être confirmé par les utilisateurs. »

Le principe fondamental : **une version livrée est un engagement**. Chaque livraison doit être accompagnée de preuves de validation, de notes de livraison complètes, d'un transfert au support et d'une décision Go / No-Go formelle.

🔀 Le passage de IMPLÉMENTER à DIFFUSER

Avant d'entrer en DIFFUSER, les critères de sortie de IMPLÉMENTER doivent être satisfaits.

Prérequis	Vérification
Toutes les stories planifiées sont implémentées	Conformément aux **spécifications IF... THEN...**
Le code respecte les standards qualité	Développement, sécurité, conformité et revues approuvées

Les tests unitaires et d'intégration passent	Couverture minimale atteinte ou dérogation approuvée
Les validations fonctionnelles confirment les **spécifications IF... THEN...**	Chaque spécification possède un résultat documenté
Les anomalies critiques ou bloquantes sont résolues	Ou documentées avec plan de résolution et dérogation approuvée
Le code est intégré et passe la CI/CD	Build vert, aucune erreur bloquante
Les artefacts de build sont générés et versionnés	Stockés dans les dépôts officiels
La documentation technique est à jour	Liée aux spécifications et aux changements livrés

Si ces prérequis ne sont pas satisfaits, on ne compense pas par une diffusion précipitée : on retourne corriger, compléter ou décider formellement d'une dérogation.

🔄 Ce que « diffuser » veut dire vraiment

DIFFUSER couvre sept activités critiques, qui ensemble rendent le produit livrable, fiable et supportable.

```
CODE VALIDÉ
   ↓
ASSEMBLER → VALIDER → PACKAGER → DÉPLOYER → DOCUMENTER → TRANSFÉRER → DÉCIDER
   ↓
BUILD CANDIDAT APPROUVÉ → CONFIRMER
```

1. 🔧 ASSEMBLER — Construire le build candidat

L'assemblage transforme le code validé en un build candidat traçable et reproductible.

Ce qu'il faut faire	Ce qu'il faut prouver
Générer le build candidat : Beta, RC, Canary ou version stable	Le build est versionné, traçable et reproductible
Produire les artefacts de diffusion	Packages, images, installeurs, scripts et migrations sont identifiés et stockés
Valider la cohérence de l'assemblage	Aucun composant manquant, aucune incompatibilité de version

Appliquer la stratégie de versionnement	La version reflète la nature du changement : majeur, mineur ou patch
Classifier la version	Production, Bêta ou Expérimentale, avec communication explicite

> « Un build candidat n'est pas un zip de fichiers. C'est un artefact versionné, traçable, reproductible et classifié. »

2. ✅ VALIDER — Prouver que le build est conforme et stable

La validation en DIFFUSER va au-delà des tests d'IMPLÉMENTER. Elle prouve que le produit assemblé fonctionne comme un tout, dans des conditions proches de la production.

Type de validation	Ce qu'on teste	Ce qu'on prouve
Tests d'intégration whitebox	Modules, API internes, composants intra-produit	Les composants assemblés fonctionnent ensemble
Tests de déploiement, packaging et installation	Installeurs, scripts, prérequis, configuration	Le produit s'installe, se configure et démarre correctement
Tests de non-régression	Fonctions existantes, parcours critiques, stabilité globale	Le produit ne dégrade pas la qualité existante
Tests de rollback et restauration	Retour arrière, désinstallation, restauration de l'état précédent	On peut revenir en arrière de manière fiable
Validation des contrats d'intégration	Interfaces, API, événements, formats avec les équipes partenaires	Les engagements inter-équipes sont respectés

Seuils de validation

Seuil	Critère
Couverture minimale	Atteinte ou dérogation approuvée
Aucun défaut bloquant ou critique	Non résolu ou non accepté formellement

Spécifications critiques validées	100 % avec au moins une validation et un résultat documenté
Indicateurs qualimétrie conformes	Dans les seuils définis
Tests de non-régression	Seuils de qualité atteints ou dérogation approuvée

3. 📦 PACKAGER — Préparer le produit pour la livraison

Le packaging transforme le build validé en produit livrable complet : pas seulement un binaire, mais tout ce qui l'accompagne.

Élément	Ce qu'il faut préparer
Installeurs et scripts	Installeurs automatisés, scripts de déploiement, migration et configuration
Prérequis documentés	Systèmes supportés, dépendances, matériel requis, licences, réseau
Profils utilisateurs	Utilisateur standard, avancé, développeur ou intégrateur
Procédure de rollback	Retour arrière documenté, testé et validé
Notes de livraison	Nouveautés, corrections, limitations connues, contournements, parties non testées
Guide d'installation	Procédures d'installation, configuration, désinstallation et dépannage initial
Changelog	Historique des changements accessible et à jour

4. 🚀 DÉPLOYER — Rendre le produit disponible

Déployer consiste à rendre le build validé et packagé accessible dans l'environnement cible, selon le processus planifié.

Ce qu'il faut faire	Ce qu'il faut vérifier
Exécuter le processus de déploiement	Automatisé, reproductible et conforme au plan

Déployer d'abord sur staging ou pré-production	L'environnement est proche de la production
Exécuter les smoke tests post-déploiement	Le produit démarre, les fonctions critiques répondent
Valider disponibilité et performance	SLA / SLO conformes aux seuils définis
Activer le monitoring et les alertes	Logs, métriques, traces et alertes opérationnels
Confirmer le rollback	Plan testé, documenté et accessible à l'équipe d'exploitation

5. 📝 DOCUMENTER — Compléter la traçabilité et la transparence

Document	Contenu	Destinataire
Notes de version internes	Détails techniques, architecture, API, risques résiduels	Dev, QA, ops, architecture
Notes de version externes	Nouveautés, corrections, limitations, contournements, parties non testées	Utilisateurs, clients, support
Documentation utilisateur	Guides, tutoriels, FAQ et changements visibles	Utilisateurs finaux
Documentation technique	API, configuration, changelog, installation	Développeurs, intégrateurs, ops
Registre des risques résiduels	Anomalies connues, limitations, dettes, contournements	Support, PO, PM, VSE
Rapport de validation	Résultats des tests, couverture, métriques, dérogations	Comité Go / No-Go, CdL, VSE

6. 🤝 TRANSFÉRER — Préparer CARE avant la livraison

Le transfert au support se fait avant la livraison, pas après. Le support ne doit jamais découvrir le produit en même temps que le client.

Ce qu'il faut transférer	Ce qu'il faut vérifier
Documentation support	Guides de dépannage, FAQ, procédures d'escalade, cas connus
Formation	Les équipes support comprennent les nouveautés, les changements et les risques
Artefacts et accès	Builds, logs, outils de diagnostic et environnements accessibles
Risques et points de vigilance	Anomalies connues, limitations et contournements communiqués
Chemin d'escalade	Défini pour les incidents bloquants, critiques et à impact client significatif
Boucle de retour	Le support sait comment remonter les signaux vers PM, PO, CdL et ÉLICITER

7. 🏁 DÉCIDER — Go / No-Go

La décision Go / No-Go est le point de contrôle final avant publication. C'est une décision formelle, documentée et fondée sur des preuves.

Critère minimal pour un Go	Preuve attendue
Couverture de validation minimale atteinte	Rapport de couverture ou dérogation approuvée
Aucun défaut bloquant ou critique non résolu	Ou accepté formellement avec dérogation
Spécifications critiques validées	100 % avec validation et résultat documenté
Notes de livraison complètes	Nouveautés, corrections, limitations, contournements, parties non testées
Transfert CARE complété	Documentation, formation, risques et escalade validés

Rollback testé et documenté	Plan de retour arrière validé
Monitoring actif	Logs, métriques, traces et alertes opérationnels

Qui décide ?

Rôle	Responsabilité
CdL	Prépare l'information, rassemble les preuves et présente le dossier
PM	Consulté pour l'impact client, produit et feuille de route
PO	Confirme que le scope livré correspond aux spécifications et à la valeur attendue
QA / Ingénieur en validation	Confirme que les seuils de validation sont atteints
VSE	Décideur final Go / No-Go sur la base des preuves recueillies

> « Un Go n'est pas un acte de foi. C'est une décision éclairée, basée sur des preuves. Un No-Go n'est pas un échec : c'est le système qui protège les utilisateurs. »

🤖 IA et automatisation pendant DIFFUSER

Levier	Application concrète
Génération automatique des notes de livraison	Compiler changements, corrections et limitations à partir des commits, stories et tests
Validation automatisée du packaging	Vérifier que tous les artefacts sont présents, versionnés et cohérents
Tests de déploiement automatisés	Déployer sur staging, exécuter smoke tests et valider les prérequis

Détection de régressions de performance	Comparer automatiquement les métriques avec la version précédente
Monitoring et alertes automatiques	Activer l'observabilité post-déploiement
Analyse de risques assistée	Croiser changements, couverture et historique de défauts pour identifier les zones à risque

👥 Qui fait quoi pendant DIFFUSER ?

Rôle	Contribution
Développeur	Assemblage, défauts de build, tests d'intégration whitebox et support aux validations techniques
QA / Ingénieur en validation	Tests de validation, seuils, rapport de conformité, recommandation Go / No-Go
Architecte	Contrats d'intégration, cohérence de l'assemblage, impacts systémiques
CdL	Coordination de la diffusion, dossier Go / No-Go, notes de livraison, communication
PO	Validation du scope livré par rapport aux spécifications et à la valeur attendue
PM	Impact client et produit, communication externe
Support / CARE	Réception du transfert, documentation support, préparation opérationnelle
Ops / Infra	Déploiement, monitoring, environnements et rollback
VSE	Décision finale Go / No-Go et arbitrage des dérogations

📊 Métriques et signaux pendant DIFFUSER

Métrique	Ce qu'elle mesure	Signal attendu

Taux de réussite des tests de validation	Conformité du build candidat	100 % des tests critiques passent
Couverture de tests	Proportion du code et des spécifications validées	Seuil minimal atteint, spécifications critiques couvertes
Défauts découverts en validation	Qualité du code entrant en diffusion	Aucun bloquant ou critique non résolu
Temps de déploiement	Efficacité du processus de déploiement	Tendance stable ou en amélioration
Succès du rollback	Fiabilité du retour arrière	Rollback fonctionnel et testé
Complétude des notes de livraison	Transparence envers les utilisateurs	Sections obligatoires remplies
Transfert CARE complété	Préparation du support	Checklist de transfert validée
Observabilité active	Capacité à confirmer après diffusion	Logs, métriques, traces et alertes en place

Les métriques recueillies pendant DIFFUSER alimentent la boucle d'amélioration continue vers ÉLICITER.

🔁 Boucles de rétroaction pendant DIFFUSER

```
IMPLÉMENTER ← défauts, régressions, incompatibilités ← DIFFUSER → métriques, signaux, risques
→ ÉLICITER
                                                        ↓
                                                   CONFIRMER
```

Signal	Action
Défaut bloquant découvert en validation	Retour à IMPLÉMENTER pour correction
Régression de performance détectée	Analyser la cause, corriger ou documenter une dérogation
Contrat d'intégration non respecté	Escalader aux équipes partenaires et bloquer si critique
Packaging incomplet ou incohérent	Corriger l'assemblage et revalider les artefacts

Transfert CARE insuffisant	Compléter documentation et formation avant publication
Métriques hors seuils	Corriger ou documenter une dérogation avec plan de retour à la conformité
Dérogation récurrente	Remonter comme signal d'amélioration à ÉLICITER

⚠️ Pièges courants à éviter

Piège	Pourquoi c'est un problème	Comment l'éviter
Réduire DIFFUSER à « faire un package »	On oublie validation, déploiement, documentation, transfert et décision	Traiter DIFFUSER comme une phase complète avec ses sept activités
Sauter les tests de déploiement	Le produit fonctionne en dev mais pas chez le client	Tester installation, configuration et rollback systématiquement
Publier sans notes de livraison complètes	Les utilisateurs ne savent pas ce qui change ni les limites connues	Exiger nouveautés, corrections, limitations, contournements et parties non testées
Ne pas tester le rollback	Impossible de revenir en arrière si un problème survient	Tester le rollback comme un test à part entière
Transférer au support après la livraison	Le support découvre le produit en même temps que le client	Compléter le transfert CARE avant le Go / No-Go
Décision Go / No-Go informelle	Personne n'est imputable et les critères ne sont pas vérifiés	Formaliser la décision avec preuves, rôles et documentation
Ne pas activer le monitoring	On ne détecte pas les problèmes post-déploiement	Activer logs, métriques, traces et alertes avant publication
Confondre DIFFUSER et CONFIRMER	La livraison n'est pas la preuve que le produit tient ses promesses	DIFFUSER rend le produit disponible ; CONFIRMER

		prouve la valeur et la réalité terrain

🔥 Politique de correctif d'urgence

Un correctif d'urgence suit un processus accéléré, mais ne contourne pas les principes fondamentaux.

Règle	Détail
Quand	Uniquement pour un incident bloquant, critique ou à impact client significatif
Approbation	CdL prépare l'information, PM consulté selon l'impact, VSE décide si requis
Périmètre	Correction ciblée, tests essentiels, notes de livraison obligatoires
Dérogation	Toute réduction du processus normal est documentée
Traçabilité	Le hotfix suit le même cycle de versionnement et de documentation

✅ Critères de sortie de DIFFUSER

- Le build candidat est assemblé, versionné, classifié et traçable.
- Les tests d'intégration, packaging, installation et non-régression sont passés.
- Le rollback est testé et documenté.
- Les notes de livraison sont complètes : nouveautés, corrections, limitations et contournements.
- La documentation utilisateur et technique est à jour.
- Le transfert CARE est complété : documentation, formation, risques et escalade.
- Le monitoring et les alertes sont activés.
- La décision Go / No-Go est formellement documentée.
- Les risques résiduels et anomalies connues sont suivis dans le backlog CARE.

🔄 Lien avec les autres phases

```
ÉLICITER → DÉFINIR → PLANIFIER → IMPLÉMENTER → DIFFUSER → CONFIRMER
```

```
                                                  ↑            |
                                      code validé ──┘            |
                                                                 |
ÉLICITER ← signaux, métriques, dérogations, améliorations ──────┘
```

DIFFUSER est la phase qui **rend réel** tout ce qui a été prévu. C'est le moment de vérité : est-ce que ce qu'on a SENSÉ, DÉFINI, PLANIFIÉ et IMPLÉMENTÉ tient ses promesses ? Si oui, on peut CONFIRMER. Si non, on corrige, on documente et on améliore.

> « On ne peut pas confirmer ce qu'on n'a pas bien diffusé. Et on ne peut pas bien diffuser ce qu'on n'a pas bien implémenté, planifié, défini et élicité. Chaque phase porte la suivante. »

Références

- Cérémonie de diffusion
- Comment implémenter — phase précédente
- Comment confirmer — phase suivante
- (SDLC) Software Development Life Cycle (officiel)
- Contrat qualité officiel OPAL-RT
- RACI — Rôles et responsabilités
- Qualimétrie officielle OPAL-RT
- SPÉCIFICATION
- Les tests (niveau × interface × angle × type)
- Exemples concrets de combinaisons
- Lexique / Terminologies officiel OPAL-RT
- Recommandations — enjeux, constats ou lacunes à résoudre / solutionner